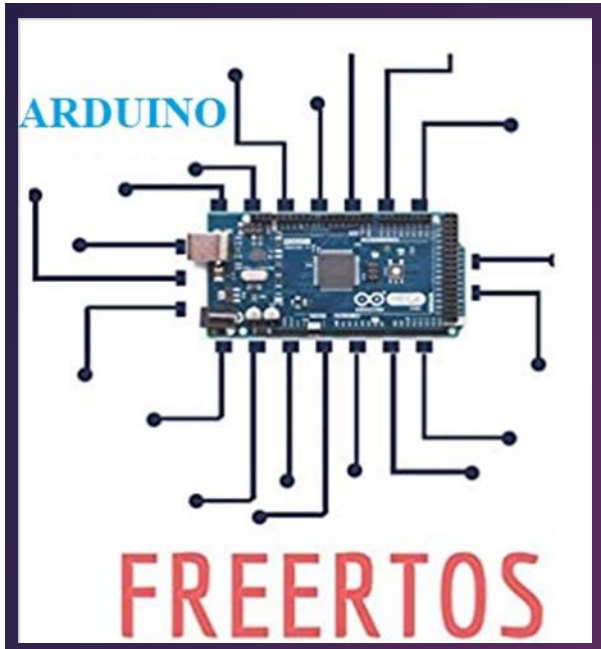


Chương 4:
Quản lý bộ định thời trong FreeRTOS

Software Timer Management

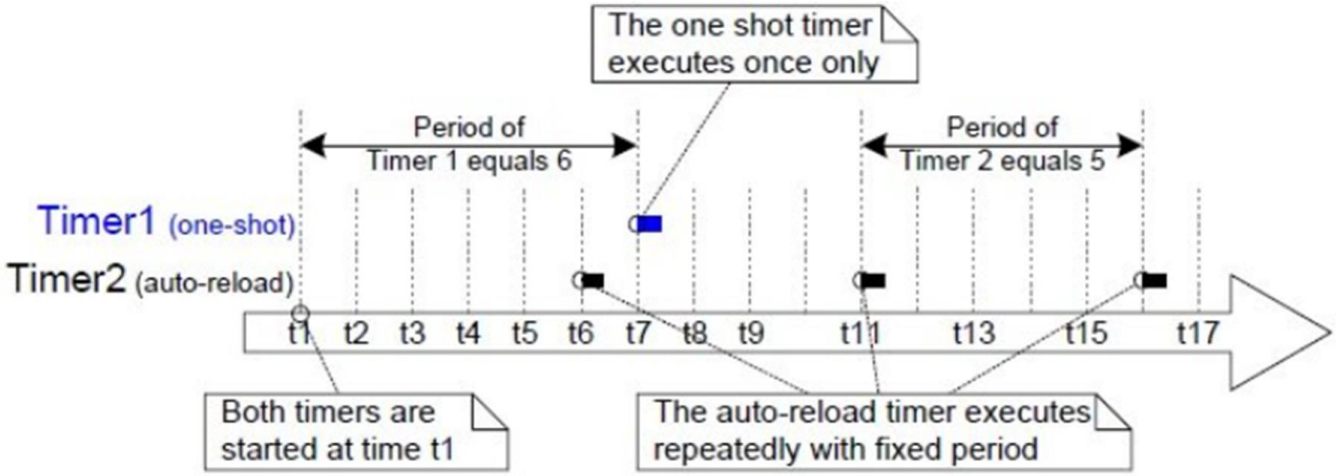
- Giới thiệu về Software Timer
- Daemon Task & Timer Command Queue
- Các thao tác trên Software Timer

Software Timer

- Bộ định thời phần mềm (Software Timer) được sử dụng để lên lịch thực hiện chức năng tại một thời điểm cụ thể trong tương lai hoặc theo các khoảng thời gian đều đặn ở một tần số cố định.
  - Chức năng chính của bộ định thời là gọi lại khoảng thời gian hẹn trước hoặc khởi động lại bộ đếm.
  - Thời gian của bộ đếm: Là khoảng thời gian giữa lúc bắt đầu bộ định thời và thực hiện chức năng gọi lại (callback) của nó.
- Trong FreeRTOS, Software Timer được sử dụng để cài đặt thời điểm thực thi của một function.
  - Mặc định Software Timer không được sử dụng, để thêm Software Timer vào chương trình, cần biên dịch thêm file ***FreeRTOS/Source/timers.c*** và set hằng số **configUSE_TIMERS** thành 1.

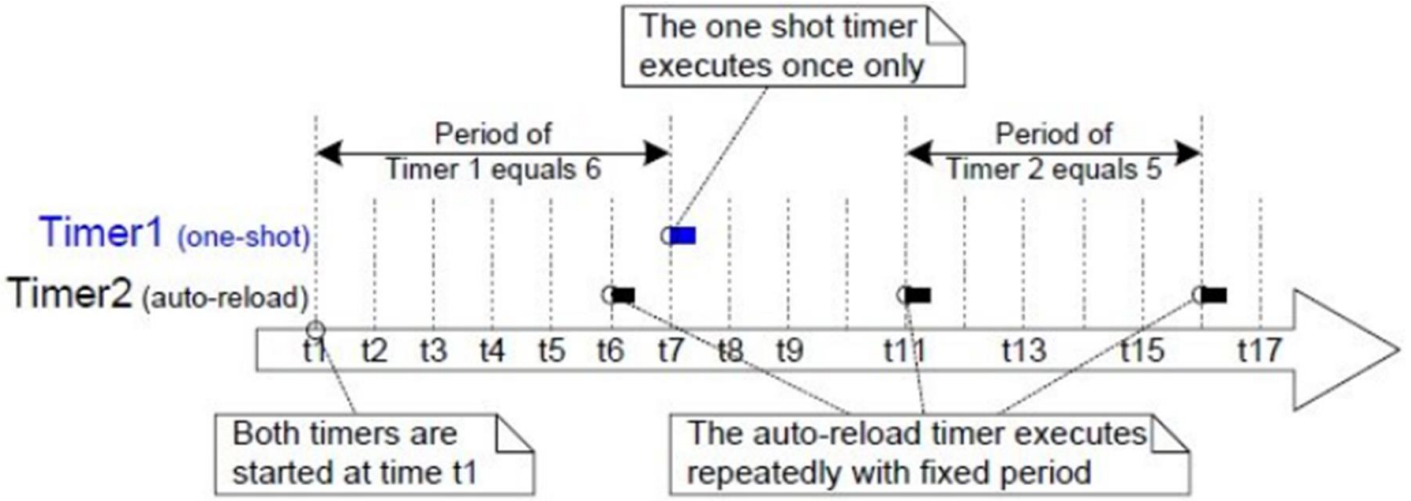
Software Timer

- Phân loại: One-shot timer và Auto-reload timer
  - One-Shot Timer (Mono-timer): Khi bộ đếm thời gian bắt đầu, nó thực hiện chức năng gọi lại một lần. Loại bộ đếm thời gian đơn này có thể được khởi động lại bằng tay và không tự khởi động lại.
  - Auto-Reload Timer(Callback-Timer): Đây là loại bộ đếm tự động, sau khi bắt đầu và kết thúc khoảng thời gian cho trước, nó sẽ tự khởi động lại theo thời gian định kỳ mà nó đã thực hiện.



Software Timer

- Phân loại: One-shot timer và Auto-reload timer
  - One-Shot Timer (Mono-timer): chỉ thực thi callback function một lần duy nhất, timer ngừng hoạt động khi đã hết hạn (expired).
  - Auto-Reload Timer(Callback-Timer): thực thi callback function khi timer expires và khởi động lại timer để tiếp tục một chu kỳ mới



Software Timer

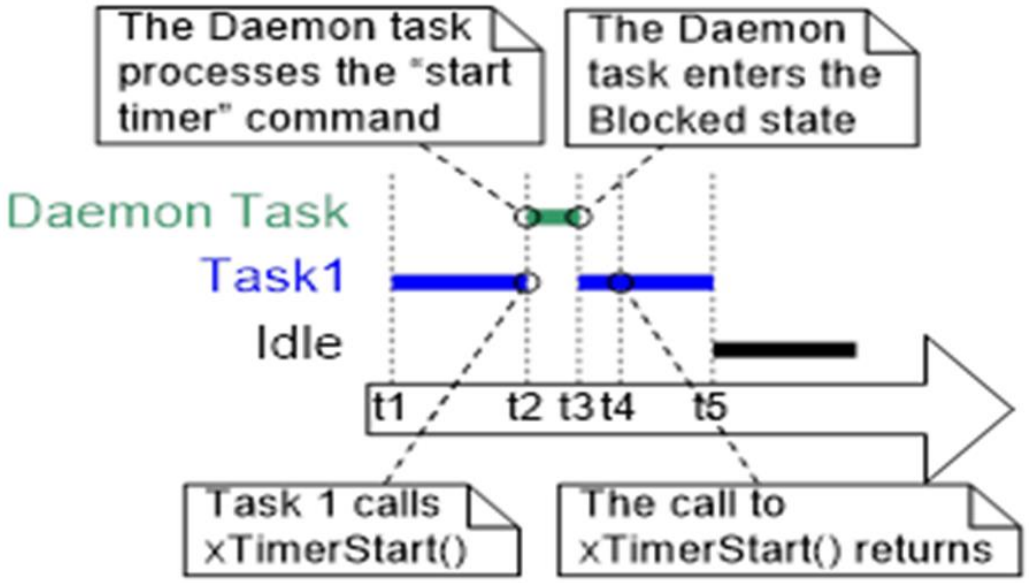
- Trạng thái của Software Timer:
  - Dormant: timer không hoạt động (ngủ đông)
  - Running: timer đang hoạt động (tích cực)
- Running: Timer đang hoạt động và thực hiện chức năng gọi lại (callback) sau khi thời gian đã trôi qua bằng thời gian của bộ đếm
- Dormant: Bộ định thời thực hiện trạng thái nghỉ, đồng thời các chức năng gọi lại thời gian của bộ đếm sẽ không được thực thi.

Software Timer Management

- Giới thiệu về Software Timer
- **Daemon Task & Timer Command Queue**
- Các thao tác trên Software Timer

Daemon Task

- Tác vụ daemon là một tác vụ FreeRTOS tiêu chuẩn được tạo tự động khi bộ lập lịch bắt đầu. Mức độ ưu tiên và kích thước ngăn xếp được đặt tại thời gian biên dịch bởi các hằng số cấu hình có sẵn trong "FreeRTOSConfig.h".
- “Daemon Task” có độ ưu tiên được quy định bởi hằng số **configTIMER_TASK_PRIORITY** và stack depth được quy định bởi **configTIMER_TASK_STACK_DEPTH**.
- Từ “Daemon Task” có thể tạo ra nhiều Software Timer.



Daemon Task

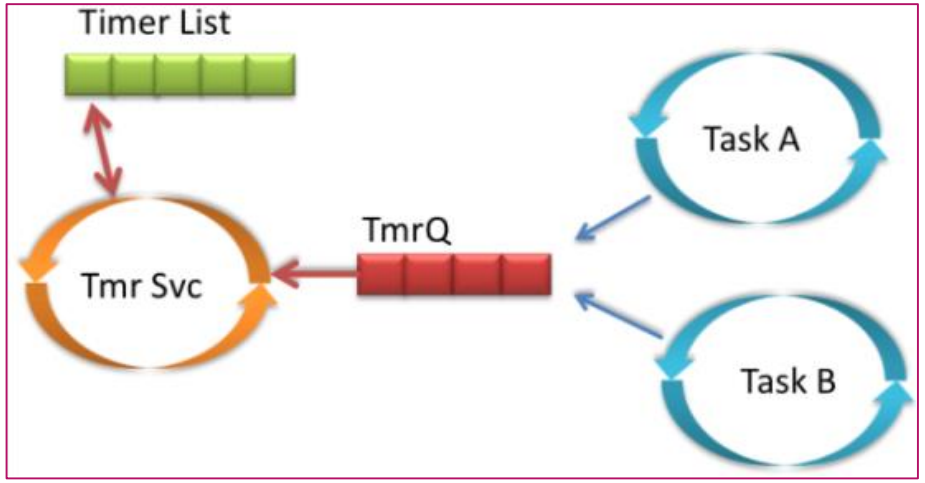
- Callback function của Software Timer sẽ được thực thi khi Timer expires, cần có nguyên mẫu như sau:

```
void timerCallback(TimerHandle_t timer);
```

- Callback function này sẽ được thực thi từ đầu đến cuối và kết thúc như các function bình thường, nó phải thật ngắn gọn và không được chứa bất cứ API nào có thể khiến “Daemon Task” rơi vào trạng thái blocked (ví dụ như vTaskDelay(), xQueueReceive() với thời gian block khác 0,… khi truy cập queue hay semaphore).

Timer Command Queue

- Vì tất cả Software Timer được quản lý bởi “Daemon Task”, bất cứ tác vụ nào nhằm thay đổi hoạt động của Software Timer (khởi động timer, reset timer, ngừng timer) đều phải được gửi tới “Daemon Task”. Để thực hiện điều này, Scheduler sẽ tự động tạo ra một queue được gọi là “Timer Command Queue” với độ lớn được quy định bởi **configTIMER_QUEUE_LENGTH**.
- “Daemon Task” sẽ rời khỏi trạng thái **blocked** nếu có command được gửi đến “Timer Command Queue” hoặc khi timer đã hết hạn để thực thi callback function.
- Cũng như những task khác, độ ưu tiên của “Daemon Task” sẽ quyết định cách mà nó được thực thi.

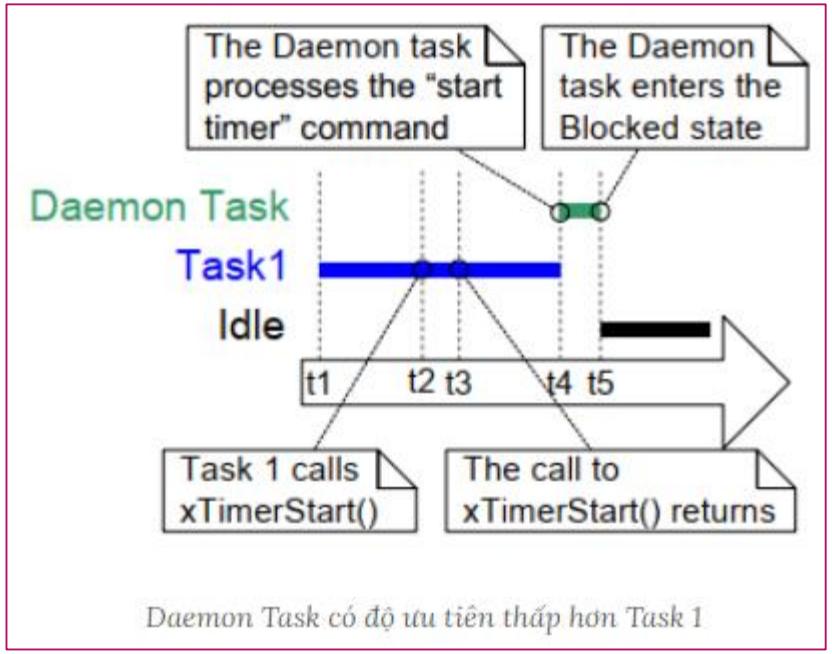



```
/* Software timer definitions. */

/* Set to 1 to include timer functionality. */
#define configUSE_TIMERS                    1
/* Sets the priority of the timer service task. */
#define configTIMER_TASK_PRIORITY           ( 2 )
/* This sets the maximum number of unprocessed commands
that the timer command queue can hold at any one time. */
#define configTIMER_QUEUE_LENGTH            5
/* Sets the size of the stack allocated to the timer
service task. */
#define configTIMER_TASK_STACK_DEPTH        ( 80 )
```

Timer Command Queue

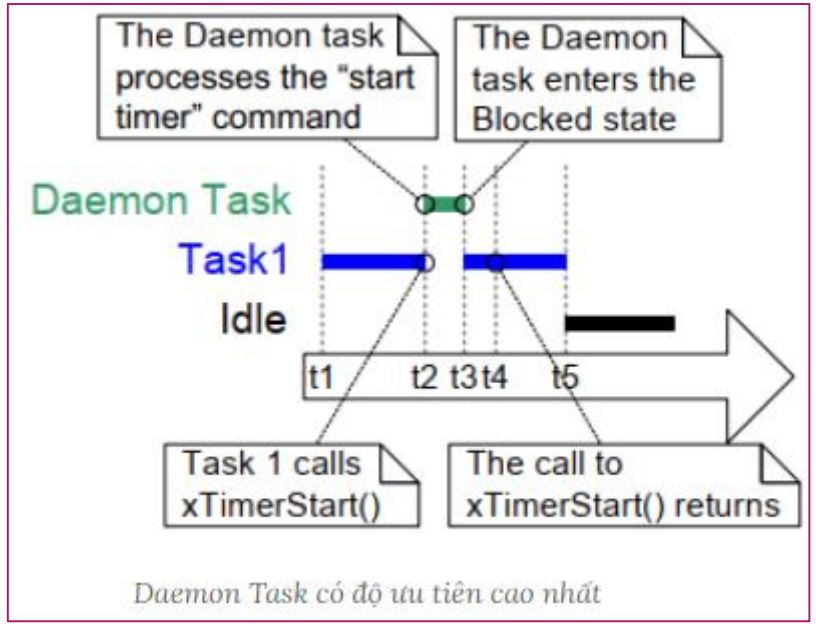
- Như ở hình bên:
  - Task 1 có độ ưu tiên cao hơn Daemon Task, vì vậy Task 1 được thực thi trước, Daemon Task ở trạng thái blocked.
  - Tại thời điểm t2, Task 1 gọi API xTimerStart() để khởi động Software Timer, Daemon Task lập tức rời khỏi trạng thái blocked và được đưa vào trạng thái Ready vì độ ưu tiên của nó thấp hơn của Task 1 nên không giành được quyền thực thi.
  - Tại thời điểm t4, Task đã hoàn thành công việc của nó và vào trạng thái blocked, Daemon Task đang ở trạng thái Ready và có độ ưu tiên cao hơn Idle Task nên được thực thi.
  - Tại thời điểm t5, Daemon Task đã xử lý xong tất cả command ở trong "Timer Command Queue" nên nó sẽ quay về trạng thái blocked, nhường quyền thực thi cho Idle Task.
- **Chú ý**: Thời điểm mà Software Timer sẽ expire được tính toán từ thời điểm mà API **xTimerStart**() được gọi, chứ không phải thời điểm mà "Daemon Task nhận được command từ "Timer Command Queue".



Daemon Task có độ ưu tiên thấp hơn Task 1

Timer Command Queue

- Như ở hình bên:
  - Daemon Task có độ ưu tiên cao nhất, Task 1 có độ ưu tiên cao hơn Idle Task.
  - Ở thời điểm t1, vì chưa có command được gửi đến nên Daemon Task vẫn ở trạng thái blocked, Task 1 được thực thi.
  - Ở thời điểm t2, Task 1 gọi API xTimerStart(), ngay lập tức Daemon Task thoát khỏi trạng thái blocked, vào trạng thái Ready và chiếm quyền thực thi của Task 1, Task 1 bị đưa vào trạng thái Ready.
  - Tại thời điểm t3, Daemon Task đã xử lý xong các command và vào trạng thái blocked, cho phép Task 1 quay lại thực thi từ trạng thái Ready.
- **Chú ý**: Mỗi command được gửi tới "Timer Command Queue" đều có chứa time-stamp ghi lại thời điểm nó được gửi, từ đó tính toán ra thời điểm mà timer sẽ expire mà không phụ thuộc vào thời điểm command được xử lý bởi Daemon Task.



Daemon Task có độ ưu tiên cao nhất

Software Timer Management

- Giới thiệu về Software Timer
- Daemon Task & Timer Command Queue
- **Các thao tác trên Software Timer**

Các thao tác trên Software Timer

- Trong bộ định thời các hàm API có vai trò quan trọng trong việc tạo, thay đổi hoặc cài đặt bộ đếm. Đây thành phần chính để chạy các chương trình của bộ định thời trong FreeRTOS
- Để tạo một Software Timer mới, ta gọi API **xTimerCreate**().
  - Software Timer có thể được tạo trước hoặc sau khi Scheduler được chạy.
  - Timer khi được tạo ra vẫn ở trạng thái Dormant.

```
TimerHandle_t xTimerCreate(const char * const timer_name,
                           TickType_t timer_period,
                           UBaseType_t auto_reload,
                           void * timer_id,
                           TimerCallbackFunction_t callbackFunction);
```

- **Trong đó:**
  - **timer_name** – tên của timer được tạo;
  - **timer_period** – chu kỳ của timer tính theo tick;
  - **auto_reload** – **pdTRUE**: tạo auto-load timer, **pdFALSE**: tạo one-shot timer;
  - **timer_id** – ID của timer, dùng để lựa chọn timer khi có nhiều timer cùng expire;
  - **callbackFunction** – callback function của Timer.
- **Giá trị trả về:**
  - **NULL** – timer không được tạo do thiếu bộ nhớ heap;
  - **non-NULL** – software timer được tạo thành công, giá trị trả về là handler của timer đó.

Ví dụ:

```
timerHndl1Sec = xTimerCreate(
      "timer1Sec", /* name */
      pdMS_TO_TICKS(1000), /* period/time */
      pdTRUE, /* auto reload */
      (void*)0, /* timer ID */
      vTimerCallback1SecExpired); /* callback */
if (timerHndl1Sec==NULL) {
  for(;;); /* failure! */
}
```

Các thao tác trên Software Timer

- Để khởi động hoặc reset một Software Timer, ta sử dụng API **xTimerStart**() hoặc **xTimerReset**().
- Timer có thể được khởi động trước khi Scheduler được chạy nhưng chỉ thật sự làm việc cho đến khi Scheduler đã bắt đầu làm việc.

```
BaseType_t xTimerStart(TimerHandle_t timer, TickType_t  ticks_to_wait);
BaseType_t xTimerReset(TimerHandle_t timer, TickType_t  ticks_to_wait);
```

- **Trong đó**:
  - **timer** – handler của software timer được trả về bởi API xTimerCreate();
  - **ticks_to_wait** – khoảng thời gian mà calling task sẽ chờ ở trạng thái blocked cho đến khi có khoảng trống ở “Timer Command Queue”.
- **Giá trị trả về:**
  - **pdPASS** – command được gửi đến “Timer Command Queue” thành công, Daemon Task sẽ được đưa vào trạng thái Ready trước khi xTimerStart() kết thúc;
  - **pdFAIL** – “Timer Command Queue” đã đầy

Các thao tác trên Software Timer

- Để thay đổi chu kỳ của Software Timer, ta sử dụng API **xTimerChangePeriod**().
  - Nếu timer đang ở trạng thái "ngủ đông" (dormant), nó sẽ tính toán lại thời điểm expire và bắt đầu hoạt động.
  - Nếu timer đang ở trạng thái Running, nó sẽ tính toán lại thời điểm expire bắt đầu từ thời điểm mà API **xTimerChangePeriod**() được gọi.

```
1  BaseType_t xTimerChangePeriod(TimerHandle_t timer,
2                                TickType_t new_period,
3                                TickType_t ticks_to_wait);
```

- **Trong đó:**
  - **timer** – handler của software timer được trả về bởi API **xTimerCreate**();
  - **new_period** – chu kỳ mới của timer tính theo tick.
  - **ticks_to_wait** – khoảng thời gian mà calling task sẽ chờ ở trạng thái blocked cho đến khi có khoảng trống ở "Timer Command Queue".
- **Giá trị trả về:**
  - **pdPASS** – command được gửi đến "Timer Command Queue" thành công, Daemon Task sẽ được đưa vào trạng thái Ready trước khi xTimerStart() kết thúc;
  - **pdFAIL** – "Timer Command Queue" đã đầy.

Các thao tác trên Software Timer

- Để thay đổi ID cho một software timer, sử dụng API **vTimerSetTimerID**().
- Để lấy giá trị ID, gọi API **pvTimerGetTimerID**().
- Khác với các API ở trên, vTimerSetTimerID() và pvTimerGetTimerID() không gửi command đến "Timer Command Queue".

```
void   vTimerSetTimerID(const TimerHandle_t timer, void *new_ID);
```

- **Trong đó:**
  - **timer** – handler của software timer được trả về bởi API **xTimerCreate**();
  - **new_ID** – ID mới của timer, phải được cast về kiểu void pointer.

```
void   *pvTimerGetTimerID(TimerHandle_t timer);
```

- **Trong đó**: **timer** – handler của software timer được trả về bởi API **xTimerCreate**().
- **Return**: ID của timer ở kiểu void pointer, cần được cast về kiểu số nguyên.

Một số API khác

- **xTimerDelete()**: Là hàm có chức năng thực thi xóa bộ hẹn giờ khi các công việc của bộ hẹn giờ không còn cần thiết trong hệ thống.
- **xTimerGetExpiryTime():** Trả về thời gian mà bộ hẹn giờ khi kết thúc, callback function sẽ thực thi.
- **pcTimerGetName()**: Trả về tên văn bản có thể đọc được do người lập trình gán khi bộ định thời được tạo.
- **xTimerIsTimerActive()**: Là hàm truy vấn bộ hẹn giờ để xác định xem bộ định thời có đang chạy hay không. Đồng hồ bấm giờ sẽ không chạy nếu:
  - 1. Bộ đếm thời gian đã được tạo, nhưng không bắt đầu.
  - 2. Bộ hẹn chỉ hẹn một lần và chưa được khởi động một lần nào nữa.
- **xTimerPendFunctionCall()**: Được sử dụng để trì hoãn việc thực thi một chức năng đối với tác vụ daemon RTOS (hay gọi là bộ đếm thời gian dịch vụ được triển khai trong timers.c).
- **xTimerStart()**: Bắt đầu hẹn giờ chạy. Hàm này sẽ tương đương với hàm resetTime khi muốn đặt lại thời gian chạy của bộ định thời.
- **xTimerStop()***:* Dừng thời gian mà bộ đếm đang chạy.

Bài tập thực hành – TH8.Timer Management

- TH8.1: Tổng hợp lý thuyết từ link (bài TH8.2) thành file word (Tiếng Việt) -> Nộp vào cuối buổi học
- TH8.2: One-shot Timer & Auto-reload Timer

https://microcontrollerslab.com/freertos-create-software-timers-with-arduino/

- TH8.3: Change Period and Reset Timer

https://microcontrollerslab.com/change-period-reset-software-timer-freertos-arduino/

Q & A